

MODULE 46

Kafka Resilience and Observability

Operate Apache Kafka reliably in production: diagnose consumer lag and failures, monitor producers, consumers, and brokers, recover from failed messages with dead letter queues, and confirm production readiness before you go live.









MODULE 46 OVERVIEW

Diagnose Kafka failures, monitor producers/consumers/brokers, recover failed messages with dead letter queues, and confirm production readiness -- with humans owning every runbook.

The Northstar CRM team's `crm.customer.events` pipeline works in staging but production Kafka fails in different ways than code bugs: lag grows silently, poison messages block partitions, brokers fail without warning.

DURATION & LAB



4 Hours Theory

Consumer lag, fault tolerance, replication, monitoring, dead letter queues, and capacity planning.



2 Hours Lab

lab46-crm: Spring Kafka error handler, DLT, idempotent projection, and observability evidence for Northstar CRM.



Scenario

`crm.customer.events` pipeline: a malformed event blocks processing while consumer lag grows unnoticed.

MODULE ARC



Understand Failure Modes

Consumer lag, backlogs, fault tolerance, replication, and high availability concepts.



Monitor & Measure

Producer, consumer, and broker metrics: throughput, latency, errors, and lag.



Recover & Harden

Dead letter queues, replay runbooks, capacity planning, and production readiness.

KEY TAKEAWAY Total time: 4 hours theory + 2 hours hands-on lab = 6 hours. Silent infinite retry on a poison message is a failing grade -- operators need lag and DLT signals with runbooks.

WHY PRODUCTION KAFKA OPERATIONS MATTER

Kafka powers mission-critical, real-time workflows -- operational discipline, not just working code, keeps it performant, available, and trustworthy.



Higher Availability

Redundant brokers and automatic failover keep the pipeline serving traffic.



Better Performance

Right-sized partitions and tuned configs sustain millions of events per second.



Data Reliability

Replication and acks settings prevent silent data loss or duplication.



Cost Optimization

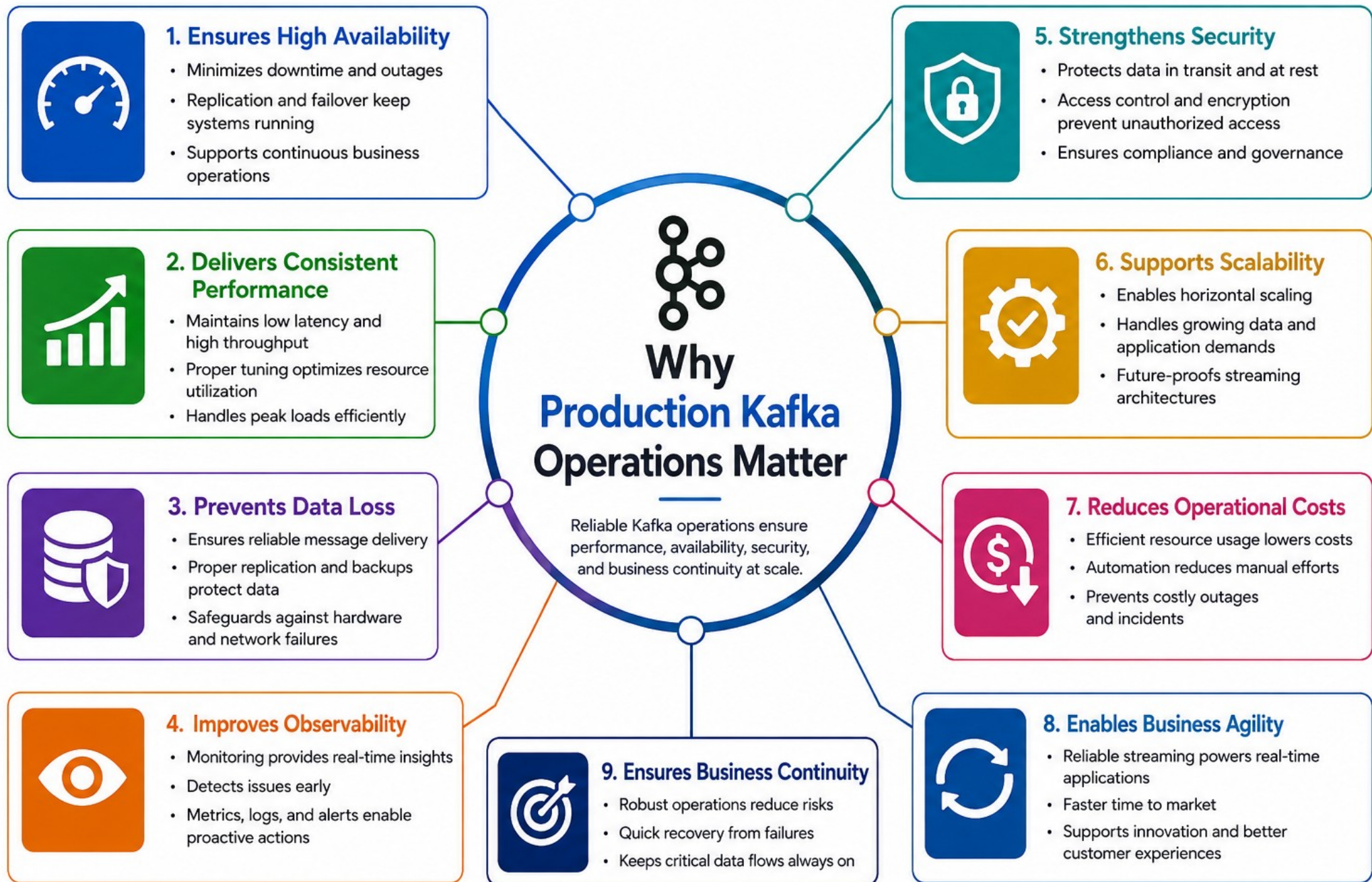
Capacity planning avoids over-provisioning while protecting headroom.



Business Confidence

Early detection via observability keeps SLAs and stakeholder trust intact.

KEY TAKEAWAY Kafka powers payments, order processing, fraud detection, and real-time analytics -- downtime and delays disrupt business operations, so resilience and high availability are non-negotiable.



MODULE LEARNING OBJECTIVES

By the end of this module, you will be able to design, operate, and monitor resilient, observable Kafka systems in production.

1. Understand Reliability Concepts -- fault tolerance, replication, and high availability.
2. Monitor Kafka Effectively -- key metrics, tools, and dashboards for producers, consumers, and brokers.
3. Analyze Consumer Lag & Backlogs -- their causes and impact on downstream systems.
4. Handle Failures Gracefully -- detect issues early and apply recovery and resilience strategies.
5. Manage Failed Messages -- implement dead letter queues and recovery strategies.
6. Improve Observability & Alerting -- logs, metrics, and alerts for operational health.
7. Plan Capacity Proactively -- handle growth and traffic spikes.
8. Apply Operational Best Practices -- secure, scalable, reliable Kafka operations.
9. Ensure Production Readiness -- work through a checklist before going live.
10. Build Real-World Skills -- hands-on labs and exercises with the Northstar CRM pipeline.

KEY TAKEAWAY You will gain practical skills to diagnose, monitor, and recover Kafka pipelines in production -- the same skills the lab grades through a real Spring Kafka error handler and DLT.

COMMON KAFKA PRODUCTION PROBLEMS

Even well-designed Kafka systems face production issues -- recognizing the pattern is the first step to preventing it.

Problem	Impact
High Consumer Lag	Delayed processing and stale data; risks SLA violations.
Message Backlog	Unprocessed messages accumulate; storage exhaustion and slow recovery.
Message Failures	Schema errors, timeouts, or bugs risk data loss or duplication.
Broker Downtime	Crashes trigger leader elections and partition unavailability.
Data Loss	Misconfigured replication or unclean shutdowns can lose data irreversibly.
Poor Observability	Missing metrics/logs/alerts means issues go undetected until impact.

OTHER FAILURE CATEGORIES TO WATCH

**Uneven Partition Load**

Hot spots and high latency from poor key distribution.

**Insufficient Resources**

CPU/memory/disk/network bottlenecks cause throttling.

**Network Issues**

Packet loss and disconnects hurt cross-AZ/region traffic.

**Duplicates & Rebalances**

Retries and frequent rebalances need idempotent handling.

KEY TAKEAWAY Monitor -> Alert -> Analyze -> Act -> Improve: catching these patterns early is cheaper than firefighting them once customers notice.

Common Kafka Production Problems

1 Broker Failures



- Hardware or OS failures
- Broker crashes or unexpected shutdowns
- Can lead to partition unavailability

 **Impact:** Downtime, unavailable partitions

2 Leader Election Issues



- Frequent leader elections
- Caused by failures or network issues
- Impacts throughput and stability

 **Impact:** High latency, reduced performance

3 Under-Replicated Partitions



- ISR smaller than replication factor
- Caused by broker failures or slow disks
- Risk of data unavailability or loss

 **Impact:** Data risk, reduced fault tolerance

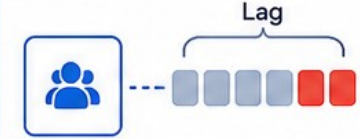
4 Disk Space Issues



- Log segments grow too large
- Insufficient disk space
- Leads to write failures and downtime

 **Impact:** Write failures, service disruption

5 High Consumer Lag



- Consumers can't keep up with producers
- Causes delayed processing and stale data
- Often due to slow processing or under-provisioning

 **Impact:** Delayed data, business impact

6 Network Problems



- Packet loss, high latency, or network partitions
- Impacts replication and client connectivity
- Can cause split-brain scenarios

 **Impact:** Errors, timeouts, partition unavailability

7 Producer Problems



- Misconfigured producers
- Acks, retries, or batch settings
- Can cause duplicates or data loss

 **Impact:** Data inconsistency, throughput issues

8 Consumer Group Rebalancing



- Frequent rebalances
- Caused by scaling or session timeouts
- Leads to processing interruptions

 **Impact:** Processing delays, reduced efficiency

9 Improper Configurations



- Wrong retention, cleanup, or segment settings
- Resource misconfiguration
- Leads to instability and data issues

 **Impact:** Instability, resource waste

10 Schema & Data Compatibility



- Incompatible schema changes
- Breaks consumers
- Causes message processing failures

 **Impact:** Errors, failed pipelines, downtime

TRY IT YOURSELF -- EXERCISE 1: FAILURE TAXONOMY

Reinforces: classifying Kafka consumer failures (validation, deserialization, transient, poison) before writing any error-handling code.

STEPS (notes/lab46-failure-taxonomy.md)

Step	What To Do
1 -- Categories	List validation, deserialization, timeout, DB, and authz failures with one CRM example each.
2 -- Reference Check	Confirm against the reference table -- poison messages must never block a partition forever.
3 -- User Impact	Map one failure to user impact: a stale CUS-1001 profile or a stuck CUS-1002 status.
4 -- Save Notes	Write the taxonomy to notes/lab46-failure-taxonomy.md in the exercises workspace.

Reference: failure category -> typical action

Validation failure	-> Dead letter topic + fix the publisher
Deserialization failure	-> Dead letter topic + schema/version check
Transient DB failure	-> Bounded retry, then dead letter topic
Poison, forever-retry	-> Forbidden pattern -- never allow unbounded retry

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 1 before continuing: exercises/exercise-01-failure-taxonomy.md (workspace: examples/module-46-exercises).

UNDERSTANDING CONSUMER LAG

Consumer lag is the difference between the latest message produced to a partition and the last message a consumer has committed.



Latest Offset

The next offset to be produced -- the log end offset.



Committed Offset

The last offset the consumer has successfully processed and committed.



Compute Lag

Lag = Latest Offset - Last Committed Offset, per partition.



Healthy = Low & Stable

A healthy lag stays low and steady -- rising lag is your earliest warning sign.

Worked example

```
Latest produced offset: 10
Last consumed offset: 7

Consumer Lag = 10 - 7 = 3 messages
```

BEST PRACTICES

- Right-size the number of consumer instances for the workload.
- Optimize consumer processing performance and batch size.
- Monitor lag continuously and set threshold-based alerts.
- Use autoscaling for consumers where the platform supports it.
- Design for idempotency so quick recovery never double-applies effects.

KEY TAKEAWAY Keep lag low and stable -- a healthy lag means your system is keeping up with real-time demand.

Understanding Consumer Lag

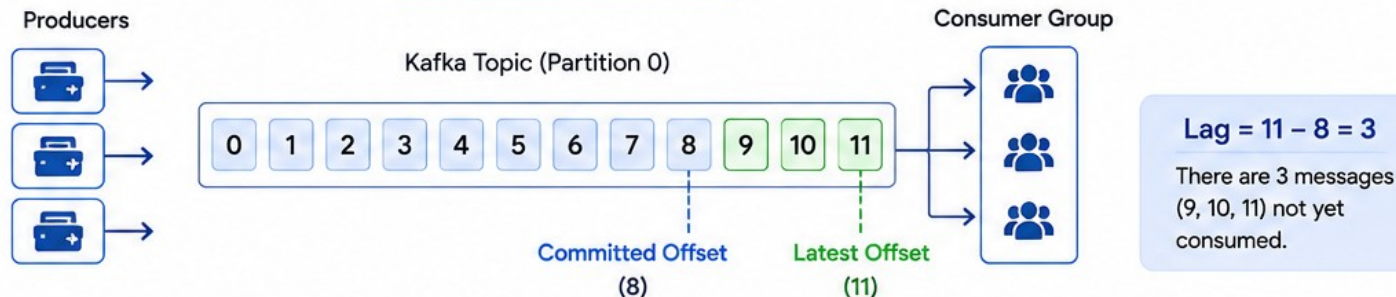
Consumer lag is the number of messages that have not yet been consumed.

1. What is Consumer Lag?

Lag = Latest Offset - Committed Offset

- **Latest Offset:** The most recent message produced to a partition.
- ➡ **Committed Offset:** The last message successfully processed and committed by the consumer.

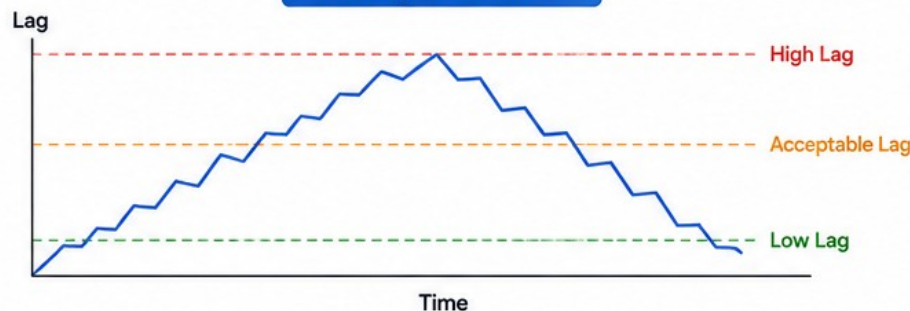
2. How Lag Happens



3. Why Lag Happens

- Consumers are slower than producers
- Insufficient consumer instances
- Long processing time per message
- Downstream system bottlenecks
- Failures and retries

4. Lag Over Time



Lag Increasing
Consumer is falling behind.



High Lag
Risk of delays and SLA breaches.



Lag Decreasing
Consumer is catching up.

5. Why Lag Matters

- High lag means delayed data processing
- Can lead to stale data in applications
- Impacts user experience and SLAs
- May indicate resource or design issues
- Long-term lag increases operational costs

6. How to Monitor & Reduce Lag



Monitor Lag
Track lag per partition, consumer group, and over time.



Scale Consumers
Add more consumer instances to increase throughput.



Optimize Processing
Improve code, reduce latency, and batch efficiently.



Right-size Resources
Ensure sufficient CPU, memory, and network capacity.



Review Architecture
Check bottlenecks in downstream systems and dependencies.

MESSAGE BACKLOGS

A message backlog occurs when messages are produced to a topic faster than consumers can process them -- the accumulation of unconsumed messages in a partition.

COMMON CAUSES

Producers sending faster than consumers can process.
Insufficient consumer instances or processing capacity.
Inefficient or slow per-message processing logic.
External system latency -- databases, APIs, downstream services.
Network or broker performance issues slowing delivery.

IMPACT

- Increased end-to-end latency across the pipeline.
- Delayed data visibility and stale reporting/dashboards.
- Higher storage usage and cost as messages pile up.
- Potential SLA violations for downstream consumers.
- System instability if the backlog keeps growing unchecked.

Monitoring dimensions

```
Consumer Lag      -- the key metric; track per group and per partition.  
Backlog Size      -- grows in step with lag.  
Oldest Message Age -- indicates how much risk is accumulating.  
Throughput        -- compare produce vs. consume rate to spot the imbalance.
```

KEY TAKEAWAY Backlogs are normal in small amounts, but growing backlogs are a warning sign -- monitor, understand the cause, and act early to keep your pipeline healthy.

Message Backlogs

A message backlog occurs when messages are produced faster than they are consumed.



1. Definition

A backlog is the accumulation of unconsumed messages in a topic partition.

It is commonly measured by consumer lag.

2. How Backlogs Form

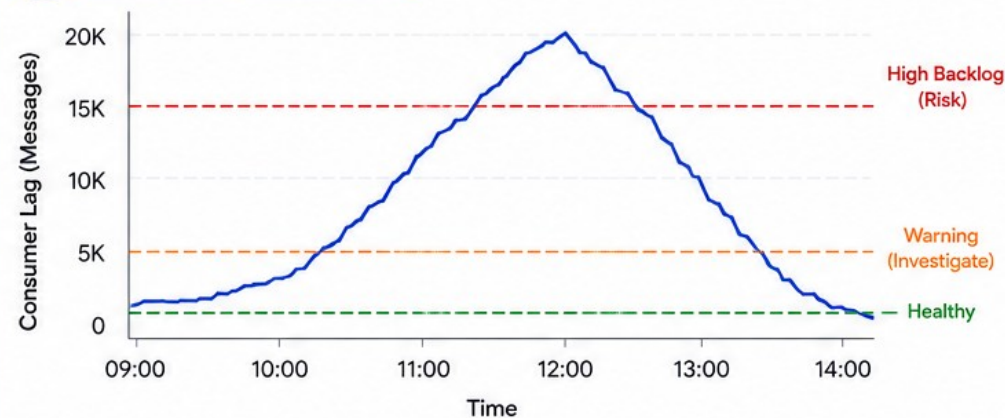


3. Common Causes

- Producers produce faster than consumers
- Insufficient consumer instances
- Slow message processing or downstream systems
- Network issues or timeouts
- Failures, restarts, or rebalancing



4. Backlog Over Time



5. Impact

- Delayed data processing and insights
- Stale data in applications and dashboards
- Increased risk of SLA breaches
- Higher infrastructure costs
- Possible data retention or overflow issues



6. Best Practices to Manage Backlogs



Scale Consumers
Add more consumer instances to increase throughput.



Optimize Processing
Improve code efficiency, reduce latency, and batch operations.



Monitor Lag
Continuously monitor lag per partition and set alerts.



Right-size Resources
Ensure sufficient CPU, memory, and network capacity.



Control Production Rate
Throttle producers or use backpressure when needed.



Use Appropriate Retention
Configure retention to handle backlogs.

FAULT TOLERANCE IN KAFKA

Kafka is designed to withstand failures and continue serving data without losing messages.



No Single Point of Failure

Data is replicated across brokers and partitions.



Partition Replication

Each partition is replicated across brokers -- default replication factor 3.



Automatic Leader Election

A new leader is auto-elected from the in-sync replicas on broker failure.



In-Sync Replicas (ISR)

Only up-to-date replicas are eligible for leader election.



Durable Storage

Messages are persisted to disk and replicated before acknowledgment.

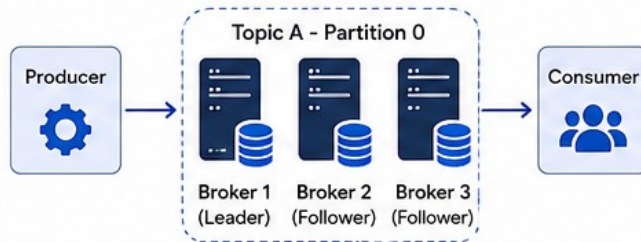
KEY TAKEAWAY If Broker 2 fails on topic orders (replication factor 3), Broker 3 is elected the new leader for its partitions -- Kafka continues serving data with no downtime.

Fault Tolerance in Kafka

Kafka is designed to continue operating and preserve data even when failures occur.

1. Built-in Replication

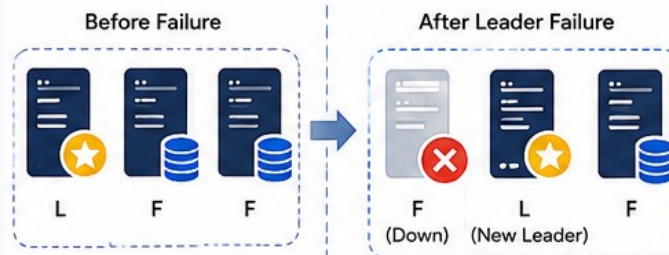
Each partition is replicated across multiple brokers.



✓ If the leader fails, a follower is automatically elected as the new leader.

2. Automatic Leader Election

When the leader broker fails, Kafka automatically elects a new leader from the in-sync replicas (ISR).



✓ No manual intervention required. Minimal service disruption.

3. In-Sync Replicas (ISR)

ISR ensures only replicas that are fully caught up can be elected leader.



✓ Ensures data consistency and prevents data loss.

4. Durable Storage

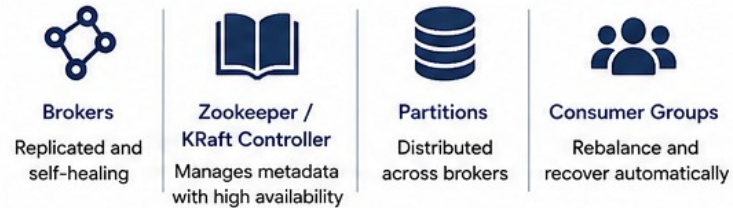
Kafka persists data to disk and uses replication to protect against data loss.



✓ Ensures messages are not lost even during crashes.

5. Fault Tolerant Components

All critical Kafka components are fault tolerant.



✓ Kafka continues to operate and recover automatically.

6. Configuration for Fault Tolerance

- ⚙️ **replication.factor**
Number of copies for each partition.
- ⚙️ **min.insync.replicas**
Minimum replicas required to acknowledge writes.
- ⚙️ **acks=all**
Ensure leader and all ISR replicas acknowledge writes.
- ⚙️ **unclean.leader.election.enable=false**
Prevent election of out-of-sync replicas.
- ⚙️ **broker.rack**
Replicate across racks/availability zones.

7. Benefits

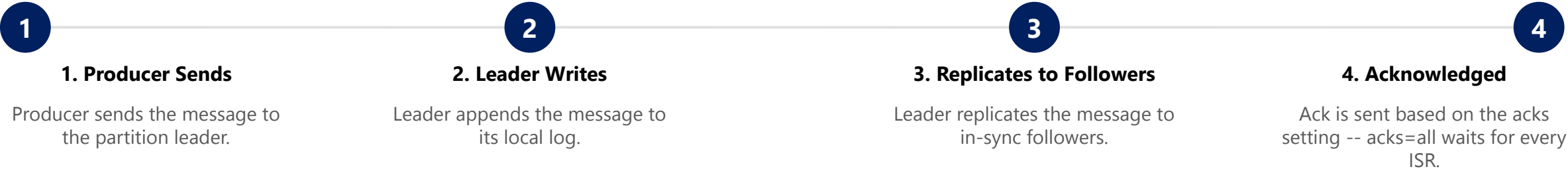
- 🛡️ **High Availability**
Continues operating despite failures.
- 🛡️ **Data Durability**
Protects data against broker or disk failures.
- 🕒 **Automatic Recovery**
Self-healing with minimal human intervention.
- 📈 **Business Continuity**
Keeps data pipelines and applications running.
- 💰 **Scalability with Safety**
Scale confidently without compromising reliability.

REPLICATION STRATEGY

Kafka replicates partition data across multiple brokers to ensure high availability and durability -- the replication factor you choose is a deliberate trade-off.

Replication Factor	Use Case	Trade-off
1	Development / testing only	No extra overhead, but zero fault tolerance -- high risk.
3 (recommended)	Production workloads	Good balance of availability and resource usage; 3x storage per partition.
5 or more	Critical systems, high durability needs	Survives more broker failures; higher storage, network, and performance cost.

WRITE PATH WITH acks=all



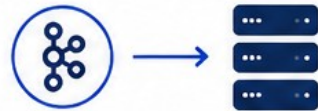
KEY TAKEAWAY Use replication factor 3 for most production workloads, and prefer an odd number so leader election during failures never ties.

Replication Strategy

Replicate data across brokers to ensure high availability, durability, and fault tolerance.

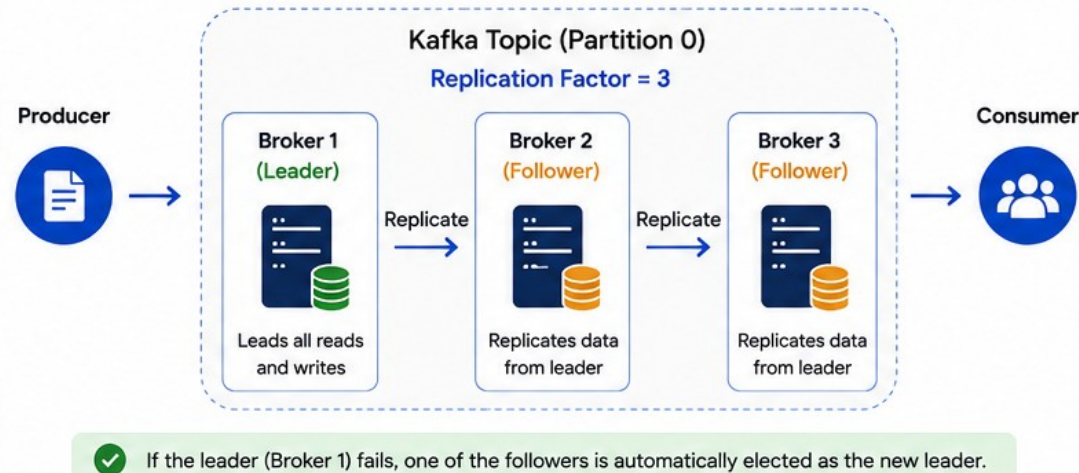
1. What is Replication?

Replication means keeping multiple copies of each partition on different brokers.



Ensures data is safe even if a broker fails.

2. How Replication Works



3. Replication Factor (RF)

Number of copies for each partition.

- RF = 1** No replication. No fault tolerance.
- RF = 2** One backup copy. Tolerates one broker failure.
- RF = 3** Two backup copies. Tolerates two broker failures.
- RF > 3** Higher durability, more storage and cost.

RF ≥ 3 is recommended for production.

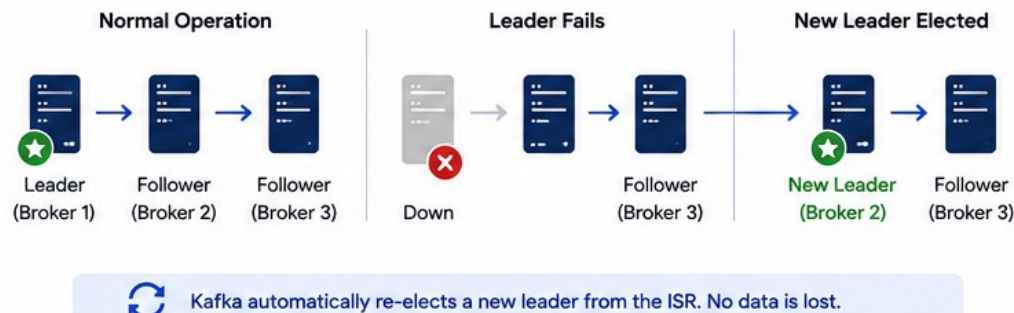
4. In-Sync Replicas (ISR)

Only in-sync replicas (ISR) are eligible to become leader.



ISR ensures strong consistency and prevents data loss.

5. Failure Scenario



6. Benefits

- High Availability** Service continues despite broker failures.
- Data Durability** Protects data from hardware failures.
- Automatic Recovery** System self-heals without manual intervention.
- Load Balanced** Distributes data and traffic across brokers.

7. Best Practices

Use RF ≥ 3 For production topics.

Place replicas on different racks/availability zones.

Monitor ISR and replication lag regularly.

Ensure sufficient disk and network capacity.

Tune `min.insync.replicas` and `acks` for reliability.

Clean up unused topics and old data with retention.

TRY IT YOURSELF -- EXERCISE 2: IDEMPOTENCY SKETCH

Reinforces: fault tolerance and replication only protect data in transit -- idempotent consumers protect against the duplicate deliveries those same mechanisms can cause.

STEPS (notes/lab46-idempotency-sketch.md)

Step	What To Do
1 -- Side Effects	List the side effects the consumer owns: projection upsert, email, audit row.
2 -- Dedupe Strategy	Pick a strategy: idempotency keys, upserts, or a dedupe store.
3 -- Duplicate Scenario	Describe a duplicate-delivery scenario for CUS-1002's status change.
4 -- Name a Test	Name one test: process the same event twice, expect exactly one projection row.

Why duplicates happen even with reliable replication

```
Producer retries after a timeout can re-send an already-delivered message.  
Consumer group rebalances can re-deliver messages near the last commit.  
At-least-once delivery is Kafka's default -- exactly-once requires extra design.
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 2 before continuing: exercises/exercise-02-idempotency-sketch.md (workspace: examples/module-46-exercises).

HIGH AVAILABILITY CONCEPTS

Kafka is designed for high availability (HA) so applications stay online even when individual components fail.



Redundancy

Data is replicated to avoid any single point of failure.



Automatic Failover

Kafka auto-detects failures and recovers service without manual steps.



No Single Point of Failure

Brokers, controllers, and storage are distributed across nodes.



Self-Healing

Kafka rebalances partitions and restores replication automatically.



Data Durability

Data stays safe through replication plus persistent storage.

KEY TAKEAWAY If Broker 2 fails, Kafka elects a new leader (Broker 3) and continues serving traffic -- no downtime, no data loss, service stays available.

High Availability Concepts

Design systems that remain operational and accessible even when failures occur.

1. What is High Availability?



High Availability (HA) ensures systems are available and operational for a very high percentage of time.



Goal: Maximize uptime, minimize downtime.

2. Key Principles



Redundancy

Eliminate single points of failure by adding backup components.



Failover

Automatically switch to a standby system when a failure occurs.



Health Monitoring

Continuously monitor components and detect issues early.



Data Protection

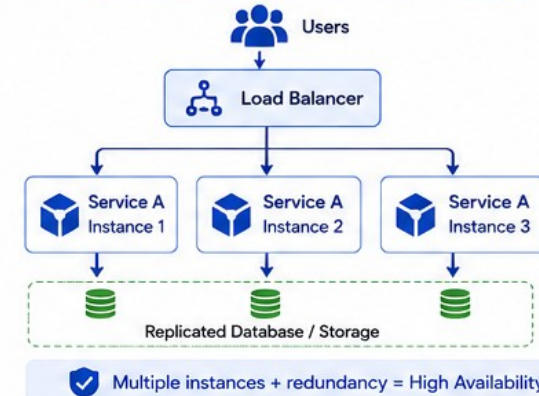
Ensure data is safe using replication, backups, and durable storage.



Automatic Recovery

System recovers automatically without manual intervention.

3. High Availability Architecture



4. Common HA Techniques



Clustering

Run multiple nodes together as a single system.



Load Balancing

Distribute traffic across multiple instances.



Replication

Keep multiple copies of data across nodes or regions.



Active-Passive

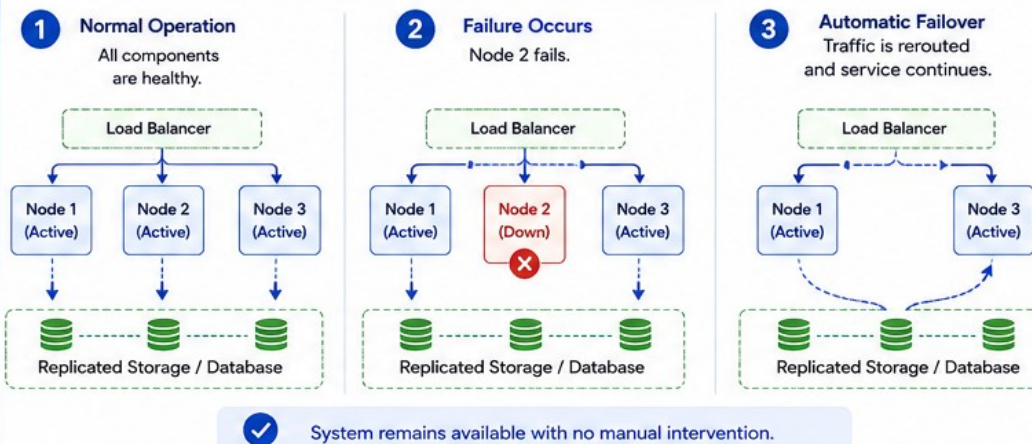
Standby system takes over when active fails.



Active-Active

All nodes are active and handle traffic simultaneously.

5. High Availability in Action (Example)



6. Benefits of High Availability



Maximized Uptime

Services remain available for users.



Fault Tolerance

Tolerates hardware, software, and network failures.



Better User Experience

Reduces downtime and service interruptions.



Business Continuity

Ensures critical operations continue without disruption.



Scalability

Easily scale to meet growing demand and improve resilience.

7. Best Practices for High Availability



Eliminate Single Points of Failure

Add redundancy at all layers.



Monitor Everything

Continuously monitor health and performance.



Test Failover

Regularly test backup and failover processes.



Backup & DR Plan

Maintain backups and disaster recovery plan.



Automate Recovery

Automate failover and recovery processes.



Multi-AZ / Multi-Region

Distribute across availability zones or regions.

KAFKA MONITORING FUNDAMENTALS

Monitoring helps you understand the health, performance, and reliability of your Kafka ecosystem so you can detect issues early and act quickly.

5 KEY MONITORING DIMENSIONS

Availability -- are brokers, partitions, and consumers healthy?

Performance -- are latency, throughput, and capacity goals met?

Reliability -- are messages replicated and delivered without loss?

Errors -- are there errors, retries, or failed requests?

Capacity -- is there enough resource headroom for current and future load?

WHAT WE MONITOR

- Brokers -- health, resource usage, network, disk, JVM.
- Topics & Partitions -- messages in/out, leader distribution.
- Producers -- throughput, errors, retries, request latency.
- Consumers -- lag, throughput, errors, commit rate.
- Cluster -- controller state, ISR, offline partitions.

Common monitoring tools

```
Prometheus -- metrics collection and time-series storage.  
Grafana -- dashboards and visualization.  
Confluent Control Center -- Kafka-native monitoring.  
Also seen: Datadog, New Relic, Elastic Stack, CloudWatch.
```

KEY TAKEAWAY Good monitoring = Early detection + Faster response + Reliable systems.

Kafka Monitoring Fundamentals

What is Monitored?

Brokers



- CPU, Memory
- Disk I/O, Network
- Log Size
- Under Replicated Partitions
- ISR Shrink/Expand

Topics & Partitions



- Partition Count
- Replication Factor
- Leader Distribution
- Offline Partitions
- Active Controllers

Producers



- Message Rate
- Error Rate
- Request Latency
- Batch Size
- Compression Rate

Consumers



- Consumer Lag
- Fetch Rate
- Commit Rate
- Request Latency
- Rebalance Count

Kafka Connect



- Connector Status
- Task Status
- Records In/Out
- Error Rate
- Lag

Monitoring Architecture

Kafka Cluster



Brokers
Topics
Producers
Consumers
Connect

Metrics Collection



JMX Exporter
or
Kafka Exporter

Metrics Store



Prometheus
Time Series DB

Visualization & Alerting



Grafana



Alertmanager

Notification Channels



Email



Slack



Webhooks



SMS / PagerDuty

Key Metrics to Monitor



Broker Health

CPU Usage, Memory Usage, Disk Usage, Network In/Out, Request Rate, Request Latency, Active Connections



Topic Health

Under Replicated Partitions, Offline Partitions, Leader Distribution, Partition Count



Producer Health

Message In Rate, Record Error Rate, Request Latency, Batch Size



Consumer Health

Consumer Lag, Commit Rate, Fetch Rate, Rebalance Rate, Last Commit Timestamp



Connect Health

Connector Status, Task Status, Records In/Out, Error Rate, Offset Lag

Best Practices



Monitor end-to-end: producers, brokers, consumers, and Connect.



Track consumer lag – the key indicator of system health.



Set up alerts for critical thresholds (CPU, lag, disk, ISR, errors).



Use dashboards to visualize trends and capacity.



Review and tune alerts regularly to reduce noise.



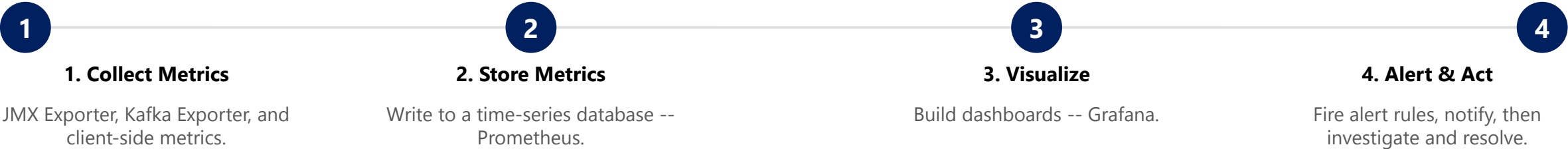
Keep monitoring stack high-availability and scalable.

PRODUCER, CONSUMER & BROKER MONITORING

Each Kafka role emits a different lens of health -- alert on all three to catch failures early.

Monitoring Target	Key Metrics & Config	Common Alerts
Producer	Send rate, success rate, error rate, request latency; acks, retries, batch.size, linger.ms.	Error rate above 1 percent; success rate below 99 percent.
Consumer	Consumer lag, throughput, commit rate, rebalance count; max.poll.interval.ms, session.timeout.ms.	Lag above 10,000 (warning); a consumer that has stopped committing.
Broker	CPU, memory, disk, ISR status, under-replicated partitions; min.insync.replicas, retention.	Disk usage above 80 percent; any under-replicated partition.

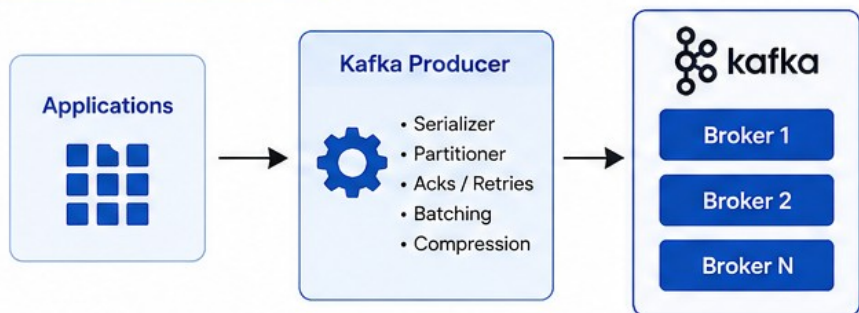
END-TO-END MONITORING FLOW



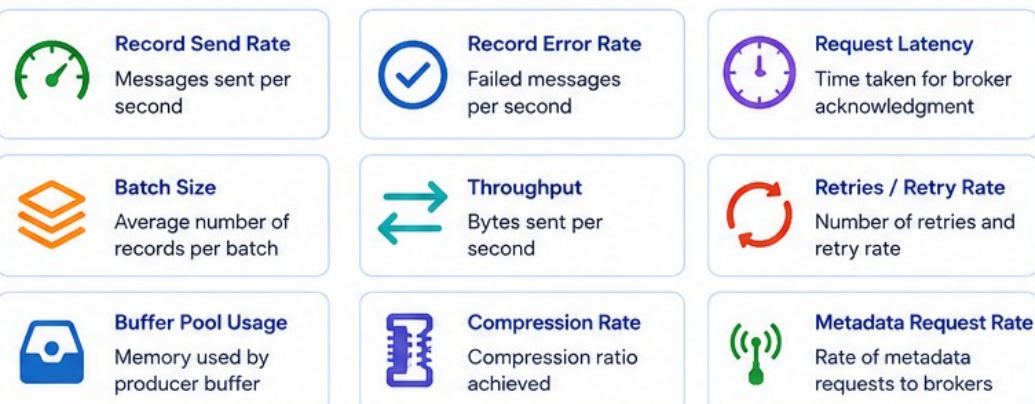
KEY TAKEAWAY Healthy producers, consumers, and brokers together mean reliable delivery end to end -- silence in one lens can hide a failure surfacing in another.

Producer Monitoring

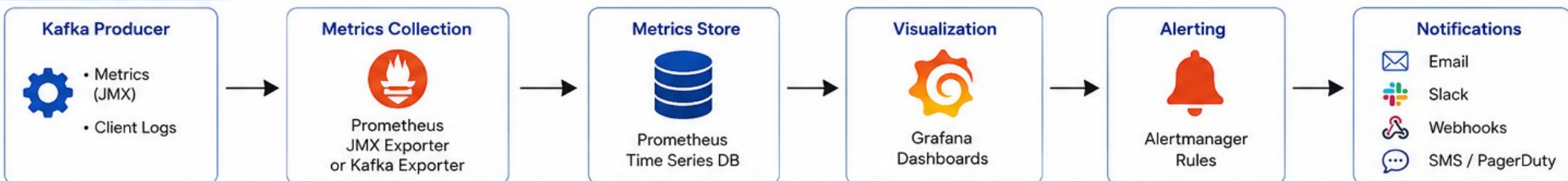
Producer Architecture



Key Producer Metrics



Monitoring Pipeline



Key Dashboards

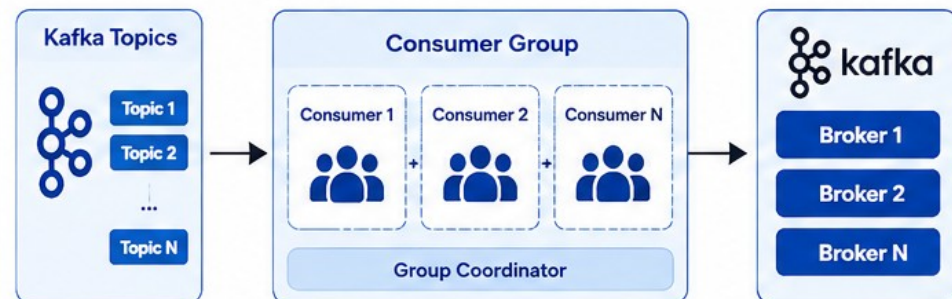


Alerting Examples

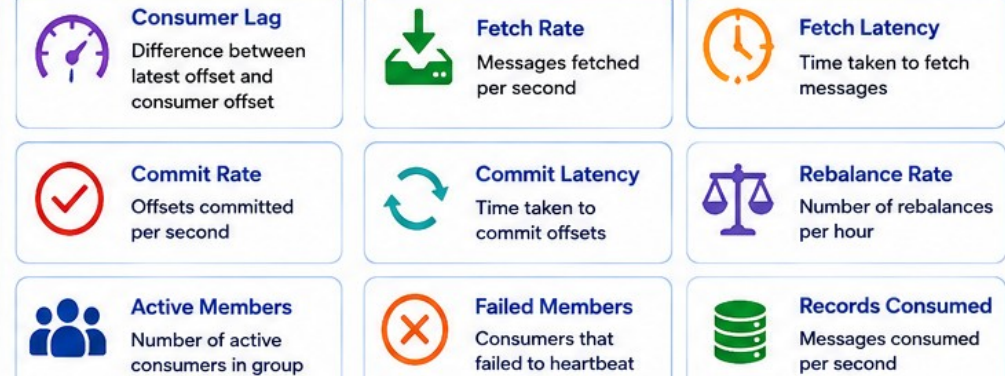
- ⚠ Record Error Rate > 0.01 for 5m
- ⚠ Request Latency (p95) > 500 ms for 5m
- ⚠ Record Send Rate drops by 50% for 5m
- ⚠ Buffer Pool Usage > 80% for 5m
- ⚠ Retry Rate > 0.05 for 5m

Consumer Monitoring

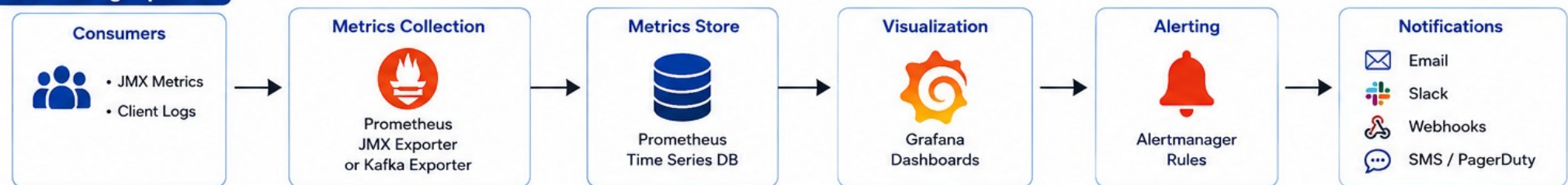
Consumer Architecture



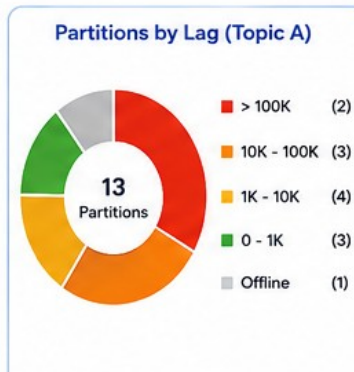
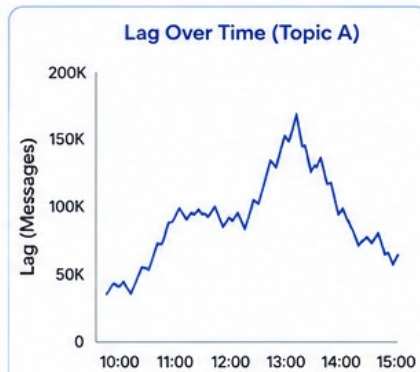
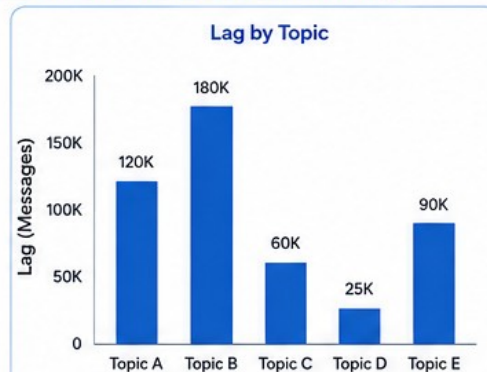
Key Consumer Metrics



Monitoring Pipeline



Consumer Lag Visualization



Alerting Examples



Broker Monitoring



Goal: Ensure high availability, performance, and reliability of Kafka brokers.



Proactive Monitoring



Reliable Platform

THROUGHPUT, LATENCY, ERROR & LAG METRICS

Four metric families tell the complete story of pipeline health -- how much, how fast, how often it fails, and how far behind.

**Throughput**
Messages and bytes in/out, per topic
-- finds hot topics.

**Latency**
End-to-end = producer + broker
queue + network + consumer
processing.

**Error**
Failed messages, timeouts,
(de)serialization and commit errors.

**Consumer Lag**
Lag = End Offset - Consumer Offset,
tracked per group.

Worked lag example

```
End Offset:      115
Consumer Offset: 108

Lag = 115 - 108 = 7 messages
```

LAG BY CONSUMER GROUP (EXAMPLE)

Group	Lag / Status
orders	8,240 -- High
payments	2,560 -- Warning
notify	950 -- Healthy
audit	0 -- Healthy

KEY TAKEAWAY Alert thresholds: lag above 10,000 = warning, above 50,000 = critical, and any sudden spike deserves immediate investigation.

Throughput Metrics



Throughput metrics measure the **amount of data successfully processed over time**. They help ensure capacity, performance, and scalability.



1. PRODUCER THROUGHPUT

- Messages In (per sec)
- Bytes In (per sec)
- Records Produced (per sec)



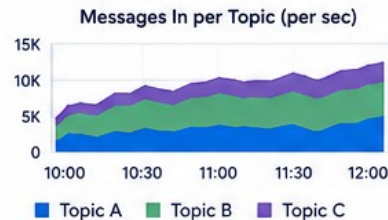
2. CONSUMER THROUGHPUT

- Messages Out (per sec)
- Bytes Out (per sec)
- Records Consumed (per sec)



3. TOPIC THROUGHPUT

- Messages In per Topic
- Messages Out per Topic
- Bytes In/Out per Topic



4. PARTITION THROUGHPUT

- Messages In per Partition
- Messages Out per Partition
- Bytes In/Out per Partition



THROUGHPUT METRICS



5. BROKER THROUGHPUT

- Bytes In (per sec)
- Bytes Out (per sec)
- Network In/Out (per sec)



6. AGGREGATE THROUGHPUT

Total Data Throughput

Messages In
120K
per sec

Messages Out
118K
per sec

Bytes In
850 MB
per sec

Bytes Out
810 MB
per sec



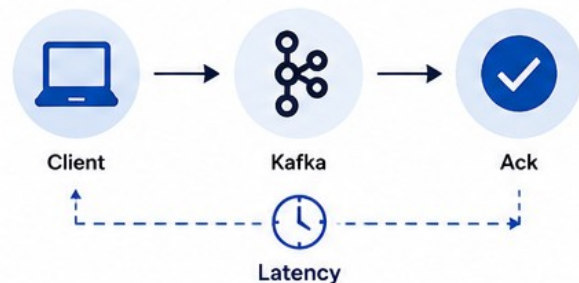
7. KEY INSIGHTS

- ✓ Monitor producer and consumer rates for balanced flow.
- ✓ Identify bottlenecks using per-topic and per-partition throughput.
- ✓ Plan capacity based on peak throughput.
- ✓ Track trends to detect anomalies early.

Latency Metrics

What is Latency?

Latency is the time taken to complete an operation from request to acknowledgement.



Key Latency Metrics



Producer Request Latency

Time from sending a produce request to receiving an acknowledgement from broker.



Consumer Fetch Latency

Time from sending a fetch request to receiving the records from broker.



End-to-End Latency

Total time from message produced by producer to consumed by consumer.



Broker Request Latency

Time taken by broker to process client request and send a response.



Commit Latency

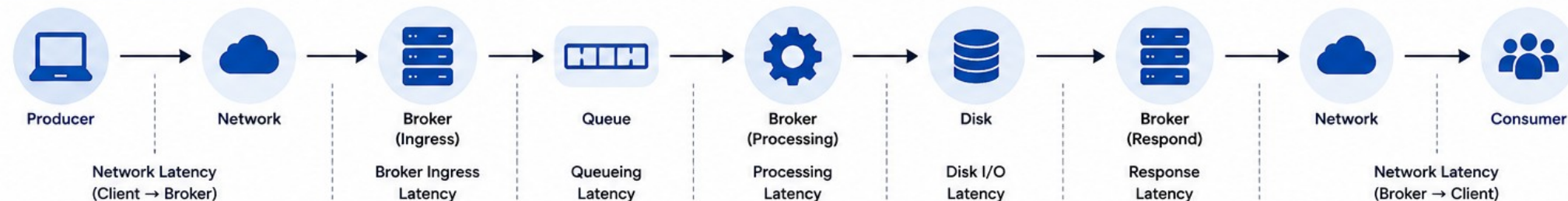
Time from sending commit request to acknowledgement from broker.



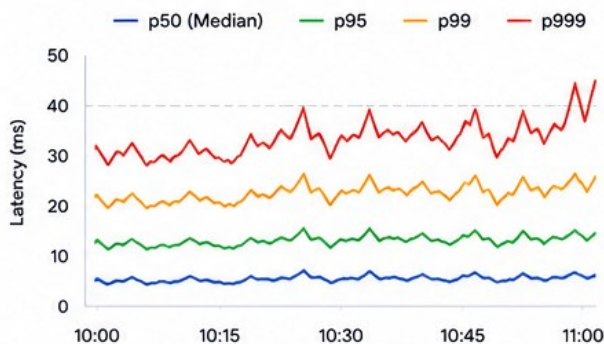
Metadata Latency

Time taken to fetch metadata from broker (topic, partitions, leaders, etc.).

Where Latency Occurs



Latency Percentiles (Example)



Typical Latency Ranges (ms)

Operation	Good (p50)	Acceptable (p95)	Poor (p99)
Producer Request Latency	< 10	10 – 50	> 100
Consumer Fetch Latency	< 10	10 – 50	> 100
End-to-End Latency	< 50	50 – 200	> 500
Commit Latency	< 20	20 – 100	> 200
Metadata Latency	< 5	5 – 20	> 50

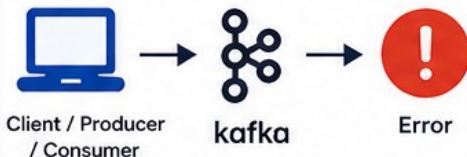
Best Practices

- ✓ Monitor latency percentiles (p95, p99, p999), not just averages.
- ✓ Investigate high tail latency (p99, p999).
- ✓ Correlate latency spikes with broker CPU, disk I/O, GC, or network.
- ✓ Use appropriate acks, batch size, and linger.ms for producers.
- ✓ Optimize consumer fetch sizes and max.poll.records.
- ✓ Keep brokers healthy and well-provisioned.
- ✓ Set alerts on latency thresholds.

Error Metrics

What are Error Metrics?

Error metrics help detect failures, data quality issues, and availability problems in Kafka.



Why it Matters

- ✓ Detect issues early
- ✓ Reduce message loss
- ✓ Improve reliability & availability
- ✓ Ensure data quality

Key Error Metrics by Component

Producer Errors



- Record Error Rate (records with errors)
- Request Error Rate (requests that failed)
- Retries / Retry Rate (retries due to failures)
- Metadata Error Rate (metadata request failures)
- Buffer Exhausted Rate (producer buffer full)

Consumer Errors



- Fetch Error Rate (fetch request failures)
- Commit Error Rate (commit request failures)
- Offset Out Of Range Rate (invalid or expired offsets)
- Deserialization Error Rate (failed to deserialize records)
- Rebalance Failure Rate (rebalance related failures)

Broker Errors



- Request Error Rate (client request failures)
- Replication Error Rate (replica fetch / sync failures)
- ISR Shrink / Expand Events (under-replication events)
- Leader Election Rate (frequent leader changes)
- Under Replicated Partitions (replication issues)

Cluster / Topic Errors



- Offline Partitions (partitions not available)
- Partition Error Rate (partition-level errors)
- Controller Error Rate (controller related failures)
- Topic Authorization Errors (authZ failures)
- Throttled Requests Rate (quota / throttling errors)

Error Monitoring Pipeline

Kafka Cluster



Producers, Brokers, Consumers, Topics, Connect

Metrics Collection



Prometheus JMX Exporter or Kafka Exporter

Metrics Store



Prometheus Time Series DB

Visualization



Grafana Dashboards

Alerting



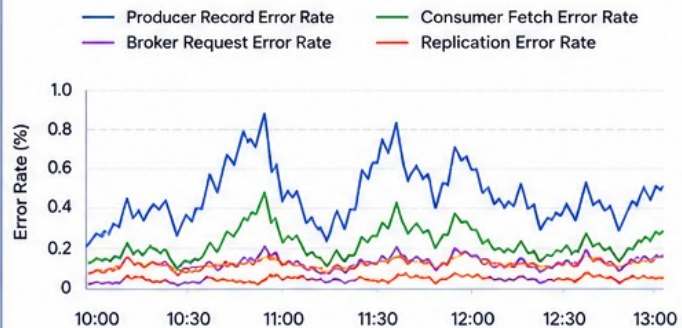
Alertmanager Rules

Notifications



Email
Slack
Webhooks
SMS / PagerDuty

Error Rate Over Time (Example)



Error Rate by Topic (Example)



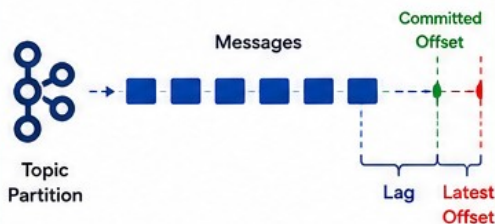
Best Practices

- ✓ Monitor error rates per component and per topic.
- ✓ Alert on both spike and sustained high error rates.
- ✓ Investigate error types and correlate with logs.
- ✓ Track error rates alongside throughput and latency.
- ✓ Ensure proper retries, timeouts, and backoff policies.
- ✓ Keep track of under-replicated and offline partitions.

Consumer Lag Metrics

What is Consumer Lag?

Consumer lag is the difference between the latest offset in a partition and the last committed offset by the consumer group.



Key Lag Metrics

Current Lag



Current number of messages the consumer group is behind the latest offset.

$$\text{Lag} = \text{Latest Offset} - \text{Committed Offset}$$

Lag by Partition



Lag for each partition in a consumer group.

Lag per Partition (partition level)

Total Lag



Sum of lag across all partitions in the consumer group.

$$\sum \text{Lag of all partitions}$$

Lag Rate



Rate at which lag is increasing or decreasing (messages / sec).

$$\text{Lag Rate} = \frac{\Delta \text{Lag}}{\Delta \text{Time}}$$

Consumer Group Lag



Overall lag for an entire consumer group.

Group Lag across topic partitions

Oldest Message Age



Time since the oldest unconsumed message was produced.

$$\text{Oldest Age} = \text{Now} - \text{Message Timestamp}$$

Lag Monitoring Pipeline

Kafka Cluster



Topics
Partitions
Consumer Groups
Offsets

Metrics Collection



Prometheus JMX Exporter
or Kafka Exporter

Metrics Store



Prometheus Time Series DB

Visualization



Grafana Dashboards

Alerting



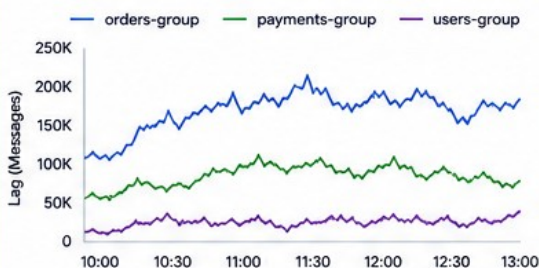
Alertmanager Rules

Notifications

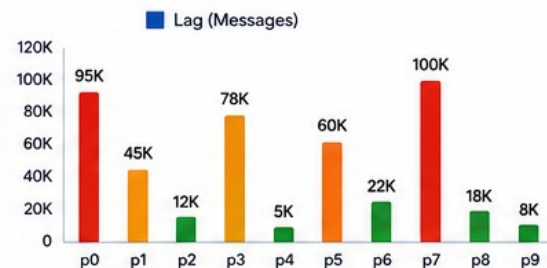


Email
Slack
Webhooks
SMS / PagerDuty

Lag Over Time (Example)



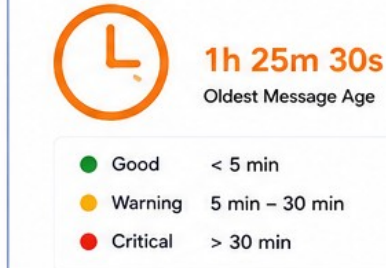
Lag by Partition (Example)



Total Lag (Example)



Oldest Message Age (Example)



Why Lag Matters



High lag indicates consumers are not keeping up.



Leads to delayed processing and stale data.



Can cause storage growth and data retention issues.



Affects SLAs and business outcomes.

Best Practices



Monitor lag per partition and total lag.



Alert on both lag value and lag rate.



Investigate root cause: slow consumers, low resources, or spikes.



Scale consumers or optimize processing.

TRY IT YOURSELF -- EXERCISE 3: METRICS & ALERT TODOS

Reinforces: turning throughput/latency/error/lag metrics into concrete dashboard panels and alert thresholds before writing any Micrometer code.

STEPS (notes/lab46-metrics-todos.md)

Step	What To Do
1 -- Fill Template	Fill processed/failed/retry/DLT counts and a lag signal with its alert threshold.
2 -- Dashboard Panels	List three panels tied to agent impact: lag, DLT growth, error rate.
3 -- Cardinality Rule	Strike any plan that tags metrics with raw customer email addresses.
4 -- Scope Check	Confirm this is a checklist only -- wiring Micrometer happens in Lab 46 itself.

Metrics/alert template to fill

```
Processed count: _____ Failed count: _____ Retry count: _____
DLT count: _____ Lag signal: _____
Alert if lag > _____ for _____
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 3 before continuing: exercises/exercise-03-metrics-todos.md (workspace: examples/module-46-exercises).

WHAT IS A DEAD LETTER QUEUE?

A Dead Letter Queue (DLQ) is a special Kafka topic that stores messages that cannot be processed successfully after a configurable number of retries.

WHY A DLQ

- Prevents a poison message from blocking the whole consumer.
- Isolates problematic messages for later analysis.
- Improves resilience and reliability of the pipeline.
- Keeps the main processing pipeline flowing.
- Enables reprocessing or manual handling after a fix.

WHAT GOES TO A DLQ

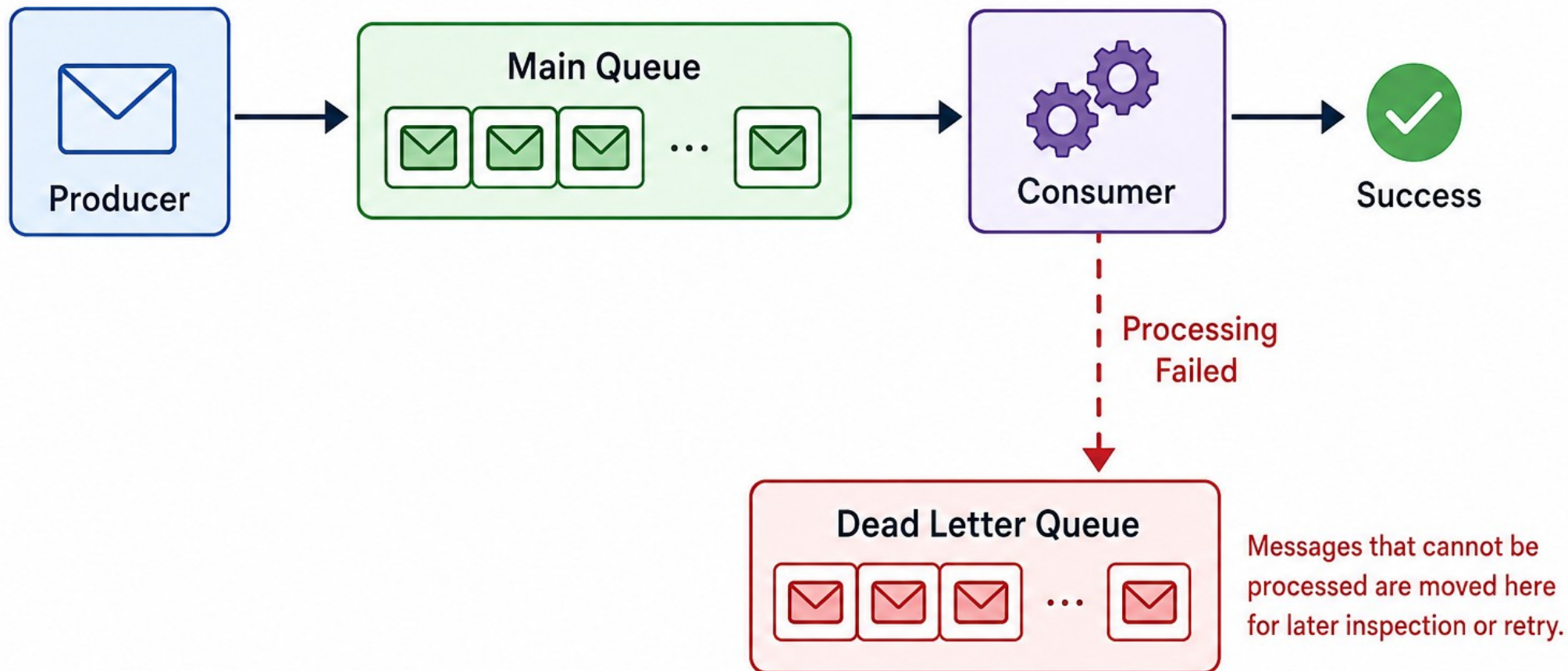
- Deserialization errors.
- Schema validation failures.
- Business rule violations.
- External service timeouts or errors.
- Unexpected runtime exceptions.

Example DLQ record

```
originalTopic: crm.customer.events      offset: 10245
error: JsonMappingException             retryCount: 3
key: order-12345   headers: content-type=application/json
```

KEY TAKEAWAY Use a separate DLQ per application or failure type, include error details and headers, and monitor and alert on DLQ size -- a growing DLQ is a growing backlog of unresolved problems.

What is a Dead Letter Queue?



TRY IT YOURSELF -- EXERCISE 4: DLT POLICY

Reinforces: naming and designing the dead letter topic policy before writing the Spring Kafka error handler in the lab.

STEPS (notes/lab46-dlt-policy.md)

Step	What To Do
1 -- Names	Name the primary topic, its DLT, and the consumer group.
2 -- Approach	Confirm the DefaultErrorHandler + DeadLetterPublishingRecoverer pattern.
3 -- Headers	List headers to preserve: original topic, exception class, correlation id.
4 -- PII Rule	State the rule: prefer customer ids over emails or names in logs and metrics.

Names to use consistently

```
Primary topic:  crm.customer.events
Dead letter topic:  crm.customer.events.DLT
Consumer group:  crm-customer-projection-v1 | Correlation:  lab-request-001
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 4 before continuing: exercises/exercise-04-dlt-policy.md (workspace: examples/module-46-exercises).

HANDLING FAILED MESSAGES

Not every message can be processed successfully -- a well-designed retry and recovery strategy keeps the pipeline reliable instead of stuck.



Immediate Retry

A fixed number of instant retries -- useful for genuinely transient errors.



Fixed Backoff

Wait a fixed interval between retries -- simple and predictable.



Exponential Backoff

Increase the wait time exponentially -- reduces load on struggling downstream systems.



Max Retry + DLT

Retry up to N times, then publish to the dead letter topic.

Failure -> Recovery flow

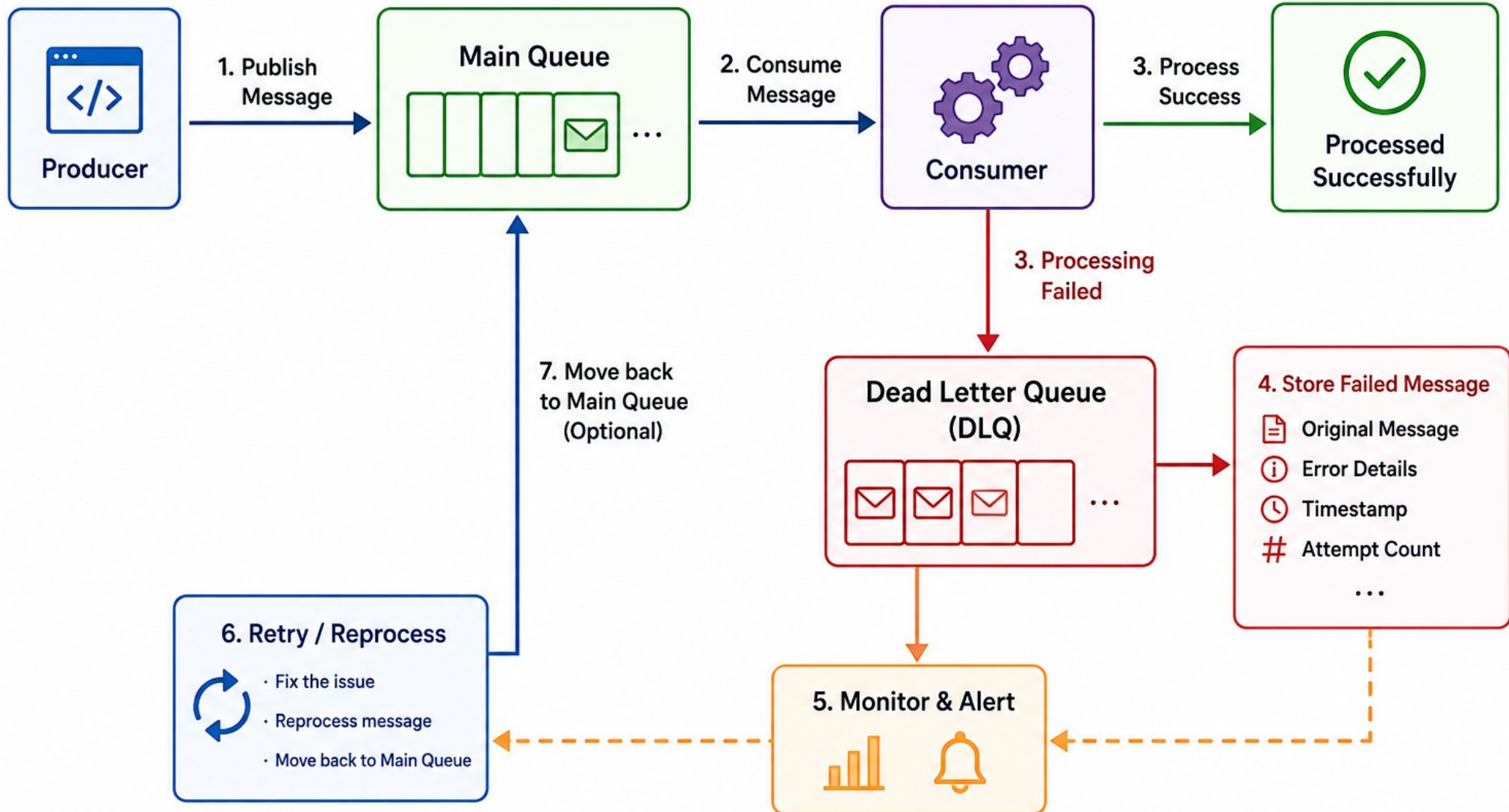
```
Consumer processes message
-> Failure -> Retry with backoff
-> Retries exhausted -> Dead Letter Topic
-> Analyze & resolve -> Reprocess
```

COMMON FAILURE CAUSES

- Deserialization errors from an invalid or corrupted format.
- Business rule violations caught during processing.
- External service failures -- timeouts, 5xx responses.
- Database errors -- connectivity or constraint violations.
- Application exceptions -- null pointers, logic bugs.

KEY TAKEAWAY Idempotent processing, sensible retry limits, and a per-application DLQ keep failures contained instead of contagious.

Handling Failed Messages



MESSAGE RECOVERY STRATEGIES

When messages cannot be processed successfully, recovery strategies let you reprocess, re-route, or fix them so no important data is lost.

Strategy	When To Use
Automatic Retry	Transient failures such as network timeouts.
DLQ + Manual Reprocess	Data issues, validation errors, business rule violations.
Fix and Re-inject	Messages can be corrected and safely reprocessed.
Route to Alternate Topic	Certain messages need a different processing pipeline.
Discard or Archive	Irrelevant, duplicate, or genuinely unrecoverable messages.

KEY METRICS TO MONITOR DURING RECOVERY



DLQ Size & Growth

Is the backlog of failed messages shrinking or growing?



Retry Attempts

Retry attempts and their success rate.



Time to Recover

How long from failure detection to resolution.

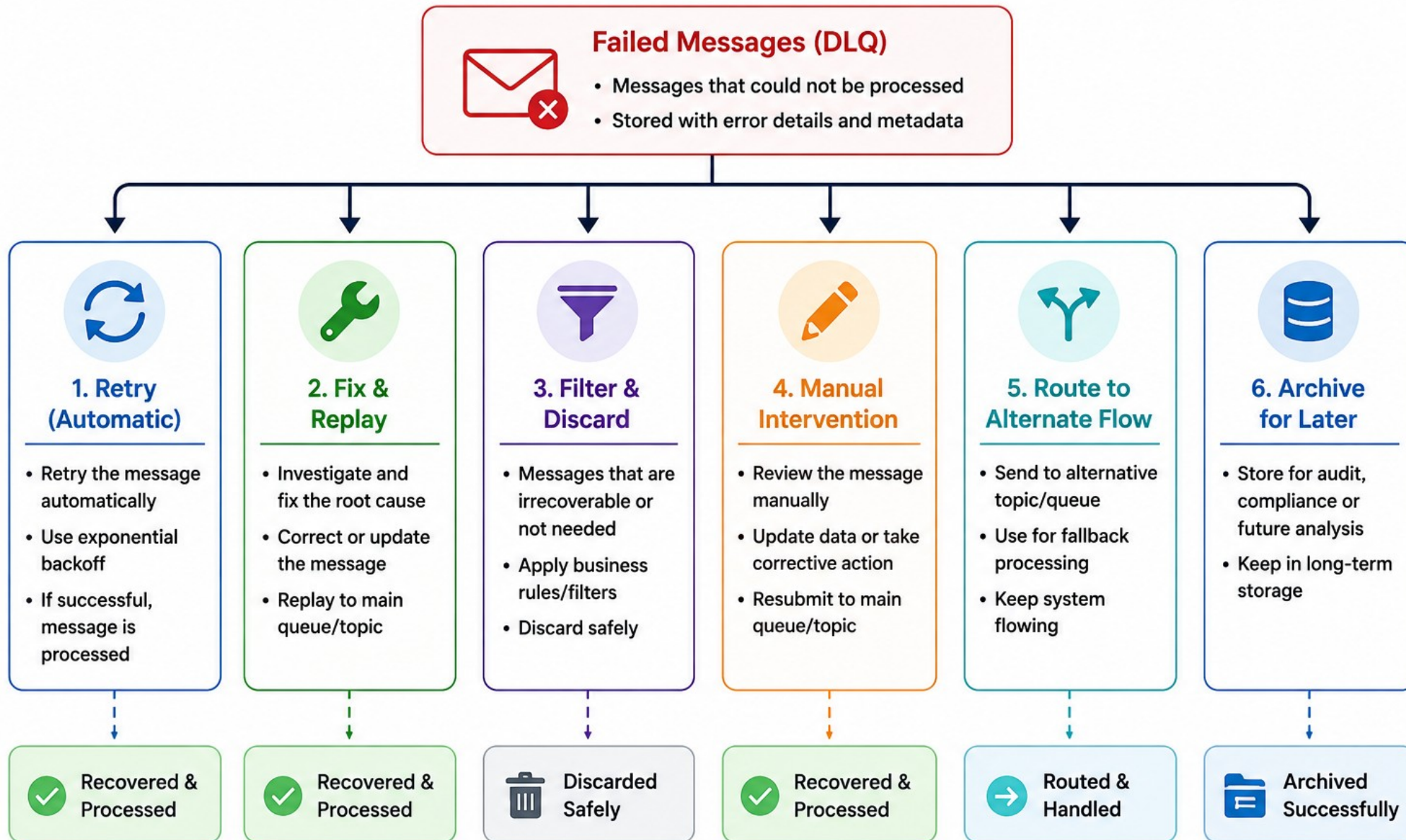


Failure Count by Reason

Which failure categories dominate the DLQ.

KEY TAKEAWAY Recovery is a diagnose-then-choose process: analyze the root cause in the DLQ, pick the matching strategy, and only then reprocess -- guessing wastes retries and risks duplicates.

Message Recovery Strategies



TRY IT YOURSELF -- EXERCISE 5: DLT REPLAY RUNBOOK

Reinforces: writing a safe, rate-limited replay procedure -- never a blind replay into production topics.

STEPS (notes/lab46-replay-runbook.md)

Step	What To Do
1 -- Sequence	Order the steps: inspect DLT, classify, dry-run, limited replay, verify, stop criteria.
2 -- Rate Limit	Confirm replay is rate-limited -- never a blind replay into production topics.
3 -- Evidence	Name the evidence proving DLT landing and a successful limited replay.
4 -- Continuity	Note that Module 47 may communicate this incident class -- keep evidence shareable.

Runbook skeleton to fill

```
1. Inspect DLT (kafka-console-consumer.sh, --print.headers=true, --max-messages 10)
2. Classify failures -> 3. Dry-run fix -> 4. Rate-limited replay -> 5. Verify projection
6. Stop criteria: pause immediately if error rate rises during replay.
```

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 5 before continuing: exercises/exercise-05-replay-runbook.md (workspace: examples/module-46-exercises).

CAPACITY PLANNING

Capacity planning ensures your Kafka platform can handle current load and future growth with optimal performance and cost efficiency.

Component	Recommended (12-Month Horizon)
Data Volume	Plan for roughly 3x growth; keep 30 percent headroom on usable disk.
Throughput	Size for projected peak, not average -- roughly 1.2x projected peak capacity.
Partitions	Grow partition count ahead of consumer parallelism needs.
Brokers	Scale broker count with data volume and throughput together.
Replication Factor	Keep at 3 for high availability regardless of growth.
Memory & CPU per Broker	Scale with workload; leave headroom, don't run at the edge.

CAPACITY PLANNING GOALS

**Meet Demand**
Handle current and forecast load without falling behind.

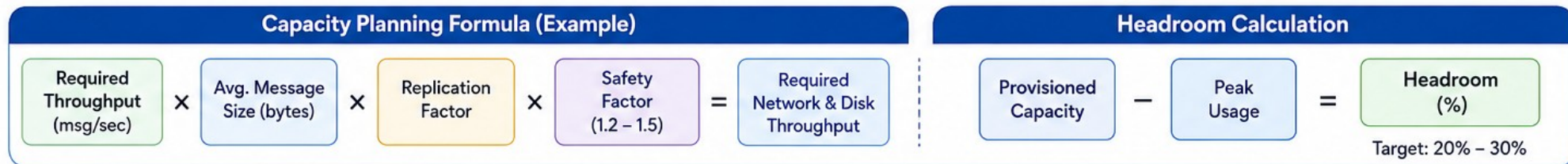
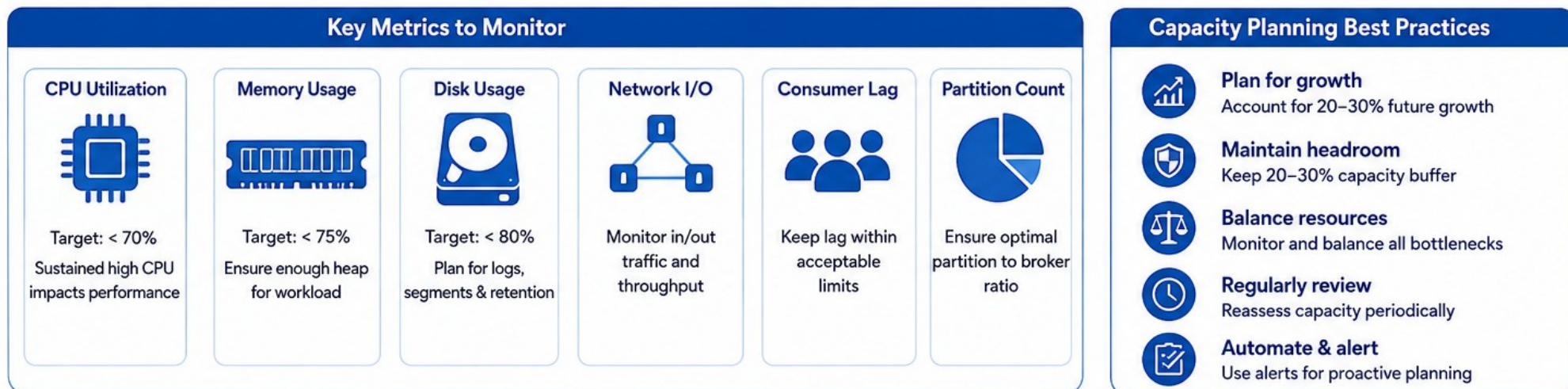
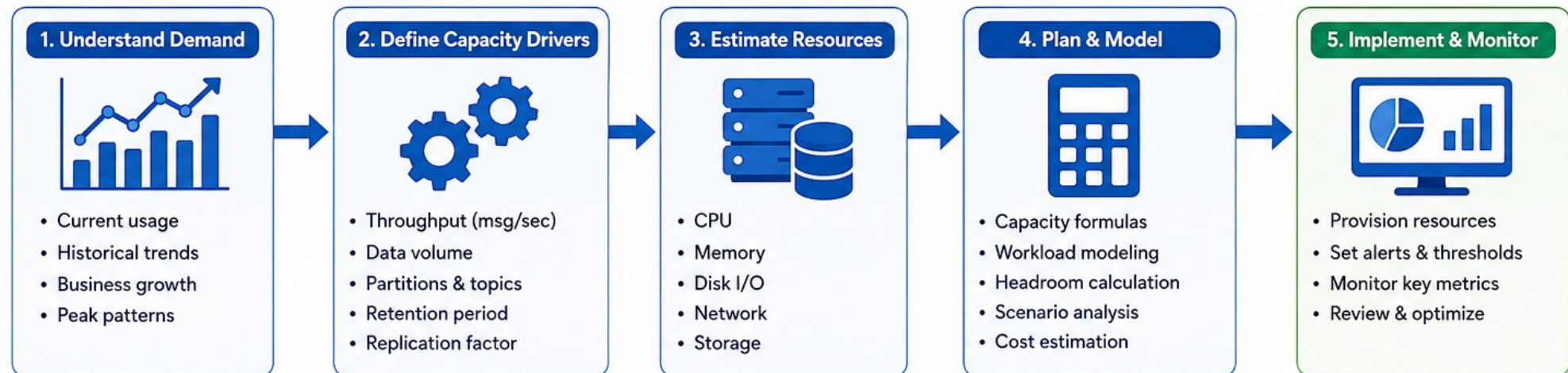
**Maintain Performance**
Avoid bottlenecks and keep latency within target.

**Optimize Cost**
Right-size resources instead of over-provisioning.

**Ensure Resilience**
Plan for redundancy, failover, and scalability.

KEY TAKEAWAY Revisit capacity plans regularly as workloads and data volumes evolve -- a plan made once and never revisited becomes wrong the moment traffic changes.

Capacity Planning



KAFKA OPERATIONAL BEST PRACTICES

Follow proven operational practices to keep your Kafka platform reliable, performant, secure, and easy to maintain.



Design for Reliability

Use replication factor 3+; enable min.insync.replicas; watch under-replicated partitions.



Plan Capacity Proactively

Monitor growth trends and maintain 20-30 percent resource headroom.



Optimize Topic Configuration

Right partition count, sensible retention, and compression where it helps.



Ensure Performance & Scale

Idempotent producers with acks=all; batch messages; scale with load.



Monitor Everything

Lag, throughput, errors, ISR, CPU, disk, network -- and act on alerts early.



Manage Consumer Groups

Meaningful group names; watch lag and rebalance events closely.



Maintain Data Integrity

Validate at producer and consumer; use a schema registry; route poison to DLQ.



Secure Your Cluster

TLS in transit, ACLs for authorization, and rotated credentials.

KEY TAKEAWAY Golden rule: automate for consistency, validate for safety, monitor for reliability, improve for the future. Common anti-pattern: manual changes made directly in production.

Kafka Operational Best Practices

1 Cluster Design & Configuration



- Use odd number of brokers (3, 5, 7...)
- Set replication.factor ≥ 3
- Tune critical configs (acks, batch.size, linger.ms, num.partitions, retention.*)
- Separate dev, test, prod environments

2 High Availability & Fault Tolerance



- Replicate topics across brokers and racks
- Enable min.insync.replicas
- Use rack awareness
- Monitor ISR shrink events
- Test broker failure scenarios regularly

3 Performance Optimization



- Right-size partitions for throughput (not too many)
- Use appropriate batch.size and linger.ms
- Enable compression (lz4/snappy)
- Tune JVM, OS, and disk I/O
- Use SSD for logs

4 Monitoring & Observability



- Monitor key metrics (JMX): throughput, latency, lag, ISR, disk, CPU, memory
- Use centralized monitoring (Prometheus, Grafana, etc.)
- Set alerts & thresholds
- Review logs and audit logs regularly

5 Data Management & Retention



- Set appropriate retention (time & size based)
- Enable log cleanup policies (delete/compact)
- Manage tiers with Tiered Storage (if applicable)
- Regularly review storage usage

6 Security Best Practices



- Enable TLS/SSL encryption
- Use SASL authentication
- Implement ACLs & least privilege access
- Rotate credentials regularly
- Audit access & permissions

7 Consumer Best Practices



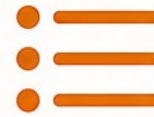
- Use consumer groups
- Commit offsets regularly
- Handle rebalances gracefully
- Monitor consumer lag
- Set max.poll.interval.ms appropriately
- Use idempotent processing

8 Producer Best Practices



- Use acks=all for durability
- Enable idempotence
- Handle retries with exponential backoff
- Monitor delivery errors
- Use meaningful keys for partitioning

9 Topic Management



- Use meaningful topic names
- Set correct partitions & replication factor
- Review and alter configs when needed
- Avoid frequent partition count changes
- Archive or delete old topics

10 Operational Readiness & Governance



- Document runbooks & SOPs
- Plan backups & disaster recovery
- Test restores and failover
- Keep Kafka and dependencies up to date
- Review capacity and plan for growth



Key Takeaway

Design for reliability, monitor everything, secure by default, and continuously optimize.

PRODUCTION READINESS CHECKLIST

Confirm your Kafka platform is reliable, scalable, secure, and observable before it goes live.

1. Architecture & Cluster Setup -- broker count, replication factor 3+, rack awareness, controller quorum.
2. Performance & Capacity -- load-tested, capacity planned, disk/CPU/memory within recommended limits.
3. Reliability & Availability -- min ISR set, unclean leader election disabled, broker failure tested.
4. Security -- TLS in transit, SASL authentication, ACLs configured and tested, secrets secured.
5. Data & Topic Management -- retention/compaction reviewed, DLQ topics created, schema registry in place.
6. Monitoring & Observability -- metrics, dashboards, alerts, and consumer lag monitoring all enabled.
7. Operations & Maintenance -- upgrade/rollback procedures and runbooks documented and version-controlled.
8. Testing & Validation -- end-to-end, failure/recovery, and load tests all passed.
9. Integration & Dependencies -- producer/consumer configuration and downstream systems validated.
10. Compliance & Governance -- audit logging, retention, access policies, and PII protection validated.

KEY TAKEAWAY All checklist items completed means the system is ready for production -- confirm the rollback strategy and inform support teams before you flip the switch.



Production Readiness Checklist



1. Functionality

- ✓ All features complete & tested
- ✓ Business requirements met
- ✓ Edge cases handled
- ✓ User acceptance completed



2. Performance

- ✓ Meets performance benchmarks
- ✓ Load & stress testing done
- ✓ Response times acceptable
- ✓ Scalability validated



3. Reliability

- ✓ High availability configured
- ✓ Failover & recovery tested
- ✓ Error handling & retries in place
- ✓ Data integrity verified



4. Security

- ✓ Authentication & authorization
- ✓ Data encryption in transit & at rest
- ✓ Vulnerability scan completed
- ✓ Secrets & keys secured



5. Observability

- ✓ Logging implemented
- ✓ Metrics collected
- ✓ Alerts configured
- ✓ Dashboards created



6. Operations

- ✓ Runbooks & SOPs documented
- ✓ Monitoring & alerting enabled
- ✓ Backup strategy in place
- ✓ Disaster recovery tested



7. Configuration

- ✓ Environment configuration ready
- ✓ Feature flags configured
- ✓ External dependencies verified
- ✓ Configuration versioned



8. Data

- ✓ Data migrations tested
- ✓ Data validation completed
- ✓ Backup & restore verified
- ✓ Data retention policies defined



9. Deployment

- ✓ CI/CD pipeline validated
- ✓ Automated deployment tested
- ✓ Rollback strategy verified
- ✓ Deployment approvals configured



10. Compliance

- ✓ Regulatory requirements met
- ✓ Audit logging enabled
- ✓ Licenses & third-party checked
- ✓ Internal policies followed



11. Documentation

- ✓ System architecture documented
- ✓ APIs documented
- ✓ Operational runbooks updated
- ✓ User guides/training ready



12. Go-Live Readiness

- ✓ Stakeholder sign-off obtained
- ✓ Go-live checklist reviewed
- ✓ Communication plan ready
- ✓ Support team prepared



FINAL GATE:
Are we ready for production?

✓ All critical items completed

✓ Risks identified & mitigated

✓ Team trained & aligned

✓ Confidence to launch



TRY IT YOURSELF -- EXERCISE 6: WATCH WINDOW

Capstone: tie observability signals to a release watch window -- the final gate before Lab 46 begins.

STEPS (notes/lab46-watch-window.md)

Step	What To Do
1 -- Watch List	List the signals to watch during a crm-api release: readiness, error rate, lag, DLT count.
2 -- Decision Link	Confirm the evidence supports a go/no-go and rollback decision, not just a status page.
3 -- Scenario	Answer: if lag spikes after 1.4.0 while agents fail on CUS-1001, what's your first check?
4 -- Save Notes	Write the watch window plan to notes/lab46-watch-window.md.

Watch-list signals for a release window

Readiness probe status	Error rate (5xx / consumer errors)
Consumer lag trend	Dead letter topic record count

KEY TAKEAWAY TRY IT NOW -- Complete Exercise 6 before continuing: exercises/exercise-06-watch-window.md (workspace: examples/module-46-exercises). Final gate before Lab 46.

LAB OVERVIEW -- KAFKA RESILIENCE FOR CRM

Build a Spring Kafka error handler, dead letter topic, and idempotent projection for the Northstar CRM event pipeline, then prove it works with observability evidence.

BUSINESS SCENARIO

- A malformed customer event repeatedly blocks processing while consumer lag grows unnoticed; agents opening Amina Khan's (CUS-1001) profile see stale data and Ravi Singh's (CUS-1002) status projection never advances.
- Poison messages must go to a dead letter topic with diagnostics; handlers must be idempotent so replay never double-applies side effects; operators need lag and DLT signals with runbooks.
- Silent infinite retry is a failing grade. Deploy Kafka with Prometheus, Grafana, and Kafdrop to monitor, detect, and recover from simulated failures end to end.

LEARNING OBJECTIVES

- Configure a DefaultErrorHandler with exponential backoff and route exhausted retries to `crm.customer.events.DLT` via a `DeadLetterPublishingRecoverer`.
- Build an idempotent projection consumer that produces the same result no matter how many times an event is replayed.
- Expose consumer lag, error, and DLT metrics through Actuator/Prometheus for Grafana dashboards.
- Simulate broker and consumer failures and observe Kafka's self-healing recovery.
- Write a DLT replay runbook and capture evidence that a limited, rate-limited replay works.
- Document configuration and observability evidence for a production readiness review.

WHAT YOU BUILD

- `lab46-crm` under `~/java-bootcamp/examples/`: a Spring Boot consumer for `crm.customer.events` with a configured error handler and DLT, an idempotent projection, Prometheus/Grafana dashboards, and `docs/kafka-dashboard.md` plus `docs/dlt-replay-runbook.md`.

KEY TAKEAWAY Fixtures CUS-1001 (Amina Khan) and CUS-1002 (Ravi Singh) are synthetic -- never real customer PII. Estimated time: 2-3 hours, Intermediate difficulty.

LAB TASKS AND DELIVERABLES

lab46-crm -- implementation steps, deliverables, and the evaluation rubric.

#	Implementation Step
1	Set up the Kafka environment: topics, partitions, replication, and the <code>crm.customer.events.DLT</code> .
2	Build the <code>DefaultErrorHandler</code> with exponential backoff and <code>DeadLetterPublishingRecoverer</code> .
3	Implement the idempotent projection consumer for <code>crm-customer-projection-v1</code> .
4	Wire Actuator/Prometheus metrics for lag, errors, retries, and DLT counts.
5	Build Grafana dashboards and document them in <code>docs/kafka-dashboard.md</code> .
6	Simulate broker and consumer failures; capture recovery evidence.
7	Inspect the DLT and perform a rate-limited replay; verify the projection updates once.
8	Write <code>docs/dlt-replay-runbook.md</code> with the diagnose-then-choose recovery sequence.

EXPECTED DELIVERABLES

- Configured error handler + DLT (`crm.customer.events.DLT`).
- DLT inspection evidence and a successful rate-limited replay.
- `docs/kafka-dashboard.md` and `docs/dlt-replay-runbook.md`.
- Failure and recovery test evidence (broker/consumer).
- Metrics and lag evidence; no secrets or real customer PII committed.

RUBRIC (100 MARKS)

- Environment/Structure 10 * Core Impl 30 * Integration/Config 15 * Failure Handling 15 * Verification 10 * Security/Production Awareness 10 * Docs 10

KEY TAKEAWAY Infinite retry or PII in logs loses security marks. Replay-all without idempotency loses recovery marks. Do not submit target/, `node_modules/`, secrets, or a verbatim solution/.

MODULE 46 SUMMARY

In this module, we explored how to build resilient, observable, and production-ready Kafka systems -- reliability concepts, monitoring strategies, and operational best practices together.

KEY CONCEPTS COVERED



Fault Tolerance

Replication, ISR, and acks keep partitions available through broker failures.



Dead Letter Queue

Capture, inspect, and replay failed messages without blocking the pipeline.



High Availability

Automatic leader election and failover mean no single point of failure.



Monitoring Basics

Track producers, consumers, brokers, and cluster health end to end.



Key Metrics

Throughput, latency, errors, and consumer lag reveal bottlenecks early.



Operational Best Practices

Capacity planning, alerting, security, and a production readiness checklist.

MODULE FLOW RECAP

1

1. Detect (Metrics)

2

2. Diagnose (Lag/Errors)

3

3. Contain (DLT)

4

4. Recover (Replay)

5

5. Validate (Readiness)

KEY TAKEAWAY Resilience plus observability equals confidence: you can now design, operate, and prove the health of a production Kafka pipeline -- not just build one that works in a demo.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of Kafka resilience, replication, and consumer lag concepts.

1. What is the primary purpose of replication in Kafka?

- A. Increase message size
- B. Improve read performance
- C. Ensure fault tolerance and high availability**
- D. Reduce storage cost

3. Which tool is commonly used to visualize Kafka metrics?

- A. Kafdrop
- B. Postman
- C. Grafana**
- D. Jenkins

2. What does consumer lag represent in Kafka?

- A. Number of consumers in a group
- B. Time taken to process a message
- C. Difference between latest and committed offsets**
- D. Size of the Kafka cluster

4. What is the purpose of a Dead Letter Queue (DLQ)?

- A. Store replicated messages
- B. Hold messages that cannot be processed successfully**
- C. Improve message throughput
- D. Backup Kafka configurations

KEY TAKEAWAY Replication ensures fault tolerance and HA; consumer lag is latest offset minus committed offset; Grafana visualizes metrics; a DLQ holds messages that failed processing.

KNOWLEDGE CHECK (2 OF 2)

Four more questions on consumer recovery, producer metrics, replication factor, and ISR.

5. Which strategy helps recover from a consumer failure?

- A. Increase partition count
- B. Use consumer group rebalancing**
- C. Delete the topic and recreate it
- D. Restart the entire cluster

7. Why should a Kafka replication factor typically be an odd number?

- A. It is easier to remember
- B. Kafka requires it by default
- C. It avoids a tie during leader election**
- D. It reduces total storage used

6. Which metric family helps identify producer performance issues?

- A. Consumer Lag
- B. Replication Factor
- C. Produce Request Rate / Errors**
- D. Partition Count

8. What does ISR stand for in Kafka?

- A. Internal Service Registry
- B. In-Sync Replicas**
- C. Independent Server Router
- D. Instance State Record

KEY TAKEAWAY Rebalancing recovers a failed consumer group; producer request rate/errors reveal producer health; odd replication factors avoid election ties; ISR means In-Sync Replicas.

MODULE 46 COMPLETE

Kafka Resilience and Observability

You can now diagnose consumer lag and backlogs, design for fault tolerance and replication, monitor producers, consumers, and brokers, recover failed messages with dead letter queues and replay runbooks, and confirm production readiness before you go live.